

Exploring the Application of Monte Carlo Neutron Transport Simulations and Parallelism

Lakota C. Dolce
Undergraduate in Computing Sciences
Coastal Carolina University
Conway, SC, USA

Abstract—This project presents a simple serial implementation of a Monte Carlo simulation for neutron transport in a homogeneous medium. The simulation models the probabilistic behavior of neutrons as they undergo reflection, absorption, or transmission through a defined region. Each neutron’s path is independently sampled using random numbers to determine free path lengths and interaction outcomes based on basic exponential and uniform probability distributions. The goal of this implementation is to illustrate the fundamental concepts of Monte Carlo particle transport and provide a baseline for further studies in performance scaling and parameter sensitivity. The program is written in C and executed serially to maintain algorithmic transparency and ease of validation. Experimental results include parameter sweeps across geometric and material parameters to explore convergence and interaction probabilities. This serial version establishes a reference for later parallel implementations using MPI and OpenMP.

Index Terms—Monte Carlo Simulation, Neutron Transport, Random, Sampling, Macroscopic Cross Section, Mean Free Path, Exponential, Attenuation Law, Pseudorandom Number Generator (PRNG), Convergence

I. INTRODUCTION

Monte Carlo methods provide a stochastic framework for modeling particle transport through matter and have become essential tools in nuclear engineering, radiation physics, detector design, and reactor analysis. Their origins trace back to the pioneering work of Ulam and von Neumann during the Manhattan Project [1], [2], where probabilistic sampling emerged as a practical strategy for studying neutron diffusion in fissionable materials. The technique formally matured into a general computational method through later contributions by Metropolis, Fermi, and Richtmyer [3], establishing the foundation of modern Monte Carlo neutron transport (MCNT).

In neutron transport, each neutron trajectory is governed by random sampling from physical probability distributions derived from macroscopic cross sections. Unlike deterministic solvers such as discrete ordinates (S_N) or diffusion theory [4], which require discretizing the neutron transport equation in angle, space, and sometimes energy, Monte Carlo techniques bypass these integro-differential complexities by simulating individual particle histories. This leads to two key benefits widely recognized in reactor physics [5]:

1) conceptual simplicity—each neutron is tracked according to fundamental physical laws; 2) geometric and material

flexibility—arbitrary shapes and heterogeneous media can be represented without stability concerns or mesh refinement.

The principal disadvantage is computational cost. Because statistical uncertainty scales as $1/\sqrt{N}$, obtaining low-variance estimates requires a large number of neutron histories. Fortunately, MCNT problems exhibit *inherent parallelism*: each neutron history is independent of every other history. This characteristic has driven decades of work on parallel transport algorithms, including the hybrid MPI/OpenMP strategies employed in production-grade codes such as MCNP [6], Serpent [7], OpenMC [8], and GEANT4 [9].

This project begins with a validated serial implementation of Monte Carlo neutron transport in a one-dimensional slab. Each neutron is launched from the left boundary ($x = 0$) and undergoes repeated cycles of free-path sampling, interaction sampling, and direction sampling until it is transmitted, reflected, or absorbed. The serial implementation computes transmission, reflection, and absorption fractions along with continuous metrics such as mean collisions and average track length. This serves as both the physical baseline and computational reference for subsequent parallel versions.

Parallelization is then pursued through two shared-memory programming models: **POSIX Threads (Pthreads)** and **OpenMP**. Each parallel version preserves the same physical kernel as the serial code. However, they differ significantly in their threading abstractions, memory semantics, load-balancing strategies, synchronization requirements, and reduction mechanisms. Because neutron histories are independent, both Pthreads and OpenMP are expected to yield substantial speedup—but the details of thread scheduling, per-thread random-number generation, and global tally reduction influence efficiency and scalability on real hardware. These effects will be quantified through analyses of speedup, efficiency, and isoefficiency using both local multicore machines and the Expanse HPC system.

A. Problem Description

We consider neutron transport through a one-dimensional slab of thickness H composed of a spatially uniform material. The material is characterized by two macroscopic cross sections:

$$\Sigma_c \text{ (absorption),} \quad \Sigma_s \text{ (scattering).}$$

The total interaction cross section is

$$\Sigma_t = \Sigma_c + \Sigma_s.$$

A neutron travels in straight-line segments between collisions. The free path ℓ to the next interaction event is drawn from the exponential probability density

$$P(\ell) = \Sigma_t e^{-\Sigma_t \ell}, \quad \ell \geq 0.$$

At the collision point, the neutron undergoes either:

$$P_a = \frac{\Sigma_c}{\Sigma_t} \quad (\text{absorption}), \quad P_s = 1 - P_a \quad (\text{scattering}).$$

Scattering is isotropic, and new direction components are uniformly distributed on the 2D unit circle. A neutron history terminates by: A neutron history terminates by:

- $x \geq H$ (transmission),
- $x \leq 0$ (reflection),
- absorption within the slab.

The simulation records the terminal fate and continuous statistics such as total track length and number of collisions.

B. Mathematical Formulas

1) *Free Path Sampling*: A uniformly distributed random number $u \in (0, 1]$ gives:

$$\ell = -\frac{\ln(u)}{\Sigma_t}.$$

2) *Interaction Type Sampling*: With $r \in (0, 1]$:

$$\text{if } r < \frac{\Sigma_c}{\Sigma_t}, \text{ absorption; otherwise scattering.}$$

3) *Isotropic Scattering*: Using a random angle $\theta = 2\pi u$:

$$\mu = \cos \theta, \quad \nu = \sin \theta.$$

C. Main Calculation (Pseudocode)

Initialize t , c , s , H , and N

Initialize tallies:

```
absorbed = reflected = transmitted = 0
total_collisions = 0
total_track_length = 0
```

For each neutron $i = 1$ to N :

```
Set (x, y) = (0, 0)
Set direction (, ) = (1, 0)
alive = true
```

While alive:

```
Sample free path = -ln(u)/t
Compute tentative position \
x_new = x +
```

```
If x_new >= H:
    transmitted++
    total_track_length += \
    boundary distance
```

```
alive = false
```

```
Else if x_new <= 0:
    reflected++
    total_track_length += \
    boundary distance
    alive = false
```

```
Else:
    total_collisions++
    total_track_length +=
```

```
Sample r
If r < c/t:
    absorbed++
    alive = false
Else:
    Sample new (, ) \
    isotropically
```

Compute averages:

```
avg_collisions = \
    total_collisions / N
avg_path_length = \
    total_track_length / N
```

D. Summary

This serial Monte Carlo neutron transport model captures the probabilistic physics of neutron behavior in a uniform slab. Terminal tallies such as absorption, transmission, and reflection emerge naturally from ensembles of random microscopic events. Because each history is statistically independent, the serial implementation provides a clean foundation for exploring shared-memory parallelization through Pthreads and OpenMP. These parallel methods allow for quantitative comparisons of speedup, efficiency, and scalability, mirroring algorithmic strategies used in modern large-scale transport codes such as MCNP, Serpent, and OpenMC.

II. RELATED WORKS

The Monte Carlo method has served as a cornerstone of particle transport simulation since its origins in the mid-twentieth century. Its development is attributed to early work by Stanislaw Ulam and John von Neumann during the Manhattan Project [1], [2]. Their insight—that complex particle transport processes could be represented through stochastic sampling—established a computational paradigm enabling neutron behavior to be modeled in geometries and material configurations that resisted analytical solutions. By simulating free-path distributions, branching interaction processes, and angular scattering using random numbers, the Monte Carlo method provided an unprecedented synthesis of physical accuracy and computational flexibility that remains central in reactor physics today [4], [5].

One of the earliest large-scale applications of Monte Carlo neutron transport was the MCN and later the MCNP (Monte

Carlo N-Particle) code family developed at Los Alamos National Laboratory [6]. MCNP pioneered continuous-energy cross-section modeling, advanced variance reduction, three-dimensional constructive solid geometry (CSG), and extensive nuclear data integration. Its accuracy and robustness made it the de facto standard for decades in shielding analysis, criticality safety, dosimetry, and reactor design. MCNP’s architectural choices—in particular, particle-based decomposition, track-length estimators, and surface/volume tallying—remain influential in modern transport codes.

More recently, a number of open-source and highly specialized Monte Carlo transport codes have continued to push the state of the art. OpenMC [8] focuses on scalability and performance portability through hybrid MPI/OpenMP models, on-the-fly temperature treatment, distributed tallies, and a domain-decomposed particle tracker. Serpent [7] emphasizes reactor analysis, featuring advanced burnup algorithms, implicit capture handling, and multi-physics coupling interfaces. GEANT4 [9], widely used beyond reactor physics, offers an extensive physics library for electromagnetic, hadronic, and optical processes, enabling applications ranging from accelerator design to medical imaging. These modern platforms demonstrate the breadth of the Monte Carlo method’s applicability and reflect decades of refinement in algorithms, data structures, and high-performance computing techniques.

Simplified pedagogical neutron transport models also play a critical role in education and verification research. Foundational texts such as Lux and Koblinger [5] and Lewis and Miller [4] present slab and one-dimensional transport problems as accessible introductions to collision sampling, exponential path distributions, and statistical estimator design. Numerous academic implementations—commonly written in C, Python, or MATLAB—adopt these simplified geometries to isolate core Monte Carlo principles. Such models allow investigation of convergence behavior, statistical uncertainty, random number quality, and performance scaling without the confounding complexity of heterogeneous geometries, resonance treatment, or cross-section processing.

Parallelization of Monte Carlo transport has also been widely studied. Owing to the statistical independence of particle histories, the Monte Carlo method is ideally suited for parallel execution, limited mainly by load balance, synchronization during tally reduction, and random number generator partitioning [10], [11]. Modern high-performance implementations often employ hybrid shared-memory and distributed-memory parallelism. MPI is commonly used for node-level decomposition, while OpenMP or POSIX threads are used for intra-node parallelism [12], [13]. OpenMC, GEANT4, MCNP6, and Serpent all incorporate variants of this hybrid strategy to achieve scalability on multi-core workstations, large NUMA systems, and leadership-class HPC platforms [14].

The present work fits within this broader lineage of educational Monte Carlo neutron transport models. The serial implementation provides a minimal yet faithful reproduction of the essential components of neutron random walks—free-path sampling, absorption and scattering events, and boundary

leakage—ensuring transparency and reproducibility. Building on this validated baseline, the project extends the core transport algorithm to shared-memory parallel versions using Pthreads and OpenMP. This mirrors the historical evolution of production-grade codes, where simple serial prototypes evolve into sophisticated, parallelized frameworks capable of handling billions of particle histories.

III. DESIGN

The design of this project integrates a validated serial Monte Carlo neutron transport (MCNT) kernel written in C with two shared-memory parallel implementations using POSIX Threads (Pthreads) and OpenMP. Python scripts perform orchestration, data collection, timing aggregation, and visualization. This structure mirrors the modular architectures used in production Monte Carlo transport systems such as MCNP [6], Serpent [7], OpenMC [8], and GEANT4 [9], which separate physics kernels from pre- and post-processing workflows.

The purpose of the design is twofold: (i) to implement a transparent serial baseline suitable for correctness validation, and (ii) to develop two independent parallelization strategies to study performance scaling, speedup, efficiency, and isoefficiency on multicore CPUs. This reflects decades of research demonstrating that Monte Carlo neutron transport is inherently parallelizable because particle histories are statistically independent [4], [5], [10].

A. System Architecture

The project is organized into three primary layers.

1) Core Transport Kernel (C):

- `mc_slab.c`: serial MCNT implementation.
- `mc_slab_pthread.c`: Pthreads-based parallel implementation.
- `mc_slab_omp.c`: OpenMP-based parallel implementation.
- `utilities.c/h`: random sampling, tally structures, JSON output, and optional trace logging.
- `timer.h`: timing macros for wall-clock measurement.

2) Analysis and Orchestration (Python):

- `sweep_mc_slab.py`: serial parameter sweeps.
- `sweep_mc_slab_pthread.py`: Pthreads sweeps.
- `sweep_mc_slab_omp.py`: OpenMP sweeps.
- `mc_slab_movie.py`: trajectory visualization from trace files.

3) *Data Products*: Runs produce JSON summaries of tallies and timing, CSV trace logs, and PNG plots for convergence, speedup, efficiency, and isoefficiency.

B. Physical and Computational Model

The physical model follows classical neutron transport in a homogeneous slab [4], [5]. The material is described by macroscopic cross sections

$$\Sigma_c : \text{absorption}, \quad (1)$$

$$\Sigma_s : \text{scattering}, \quad (2)$$

with total cross section

$$\Sigma_t = \Sigma_c + \Sigma_s. \quad (3)$$

The distance ℓ to the next interaction is sampled from an exponential distribution:

$$\ell = -\frac{\ln(u)}{\Sigma_t}, \quad u \in (0, 1]. \quad (4)$$

At the interaction point, a uniform random number r determines the event:

$$\text{absorption if } r < \frac{\Sigma_c}{\Sigma_t}, \quad \text{otherwise scattering.} \quad (5)$$

For isotropic scattering in two dimensions,

$$\theta = 2\pi u, \quad \mu = \cos \theta, \quad \nu = \sin \theta. \quad (6)$$

A neutron history terminates when

- $x \geq H$ (transmission),
- $x \leq 0$ (reflection), or
- absorption occurs inside the slab.

C. Serial Algorithm

The serial implementation in `mc_slab.c` establishes the baseline for correctness and performance.

Initialize `t`, `c`, `s`, `H`, and `N`

```
Tallies = {absorbed=0, reflected=0,
            transmitted=0,
            total_collisions=0,
            total_track_length=0}
```

```
For i in 1..N:
    x, y = 0, 0
    direction (, ) = (1, 0)
    alive = true

    while alive:
        = -ln(u)/t
        x_new = x +

        if x_new >= H:
            transmitted++
            total_track_length += \
                (H - x)
            alive = false

        else if x_new <= 0:
            reflected++
            total_track_length += x
            alive = false

        else:
            total_collisions++
            total_track_length +=

            if r < c/t:
```

```
            absorbed++
            alive = false
        else:
            sample new (, )
            x = x_new
```

At the end of the run, averages are computed:

$$\text{avg_collisions} = \frac{\text{total_collisions}}{N}, \quad (7)$$

$$\text{avg_path_length} = \frac{\text{total_track_length}}{N}. \quad (8)$$

D. Parallel Design Using Pthreads

The Pthreads implementation in `mc_slab_pthread.c` uses explicit thread creation and synchronization [10], [13]. The total number of histories N is partitioned into p blocks, one per thread:

$$N_t \approx \left\lceil \frac{N}{p} \right\rceil. \quad (9)$$

Each thread maintains its own local tallies and random number generator state. A simple seeding scheme is used:

$$s_t = s_0 + c \cdot t, \quad (10)$$

where s_0 is a base seed, t is the thread index, and c is a large odd constant to reduce correlation.

Pthreads Pseudocode:

Main:

```
init global tallies and mutex
create p threads, each with a block \
    of histories
join all threads
compute averages, write JSON
```

Thread function:

```
init local tallies and RNG
for each assigned neutron:
    run serial transport loop
    update local tallies
```

```
lock(mutex)
add local to global tallies
unlock(mutex)
```

Synchronization occurs only in the final reduction, so the inner transport loop remains lock-free and scalable.

E. Parallel Design Using OpenMP

The OpenMP implementation in `mc_slab_omp.c` employs a directive-based shared-memory model [10], [12]. The history loop is parallelized with a work-sharing construct, and tallies are combined via reduction clauses.

Instead of a long unbreakable directive, we refer to a compact form:

- A typical directive is `#pragma omp parallel for with a reduction over tallies, e.g. reduction(+:A,R,T).`

OpenMP Pseudocode:

```
A = R = T = 0    # tallies

#pragma omp parallel for reduction(+:A,R,T)
for i in 1..N:
    simulate one neutron history
    update A, R, T locally (reduced at end)
```

Here, A , R , and T denote absorbed, reflected, and transmitted counts. Thread-private variables (position, direction, RNG state) are declared to prevent data races. OpenMP manages the thread pool, loop scheduling, and reduction mechanics, reducing code complexity relative to Pthreads.

F. Python-Based Data Collection

The Python scripts automate sweeps over slab thickness H , thread count p , and number of histories N and collect JSON outputs from all three implementations. The sweep scripts:

- run the appropriate C binary,
- parse the JSON summary (tallies and runtime),
- append results to CSV tables,
- generate PNG plots for:
 - transmission, reflection, absorption vs. H ,
 - average collisions vs. H ,
 - speedup $S(p) = T_1/T_p$,
 - efficiency $E(p) = S(p)/p$,
 - empirical isoefficiency curves.

G. Extensibility

Because the physics kernel is shared across the serial, Pthreads, and OpenMP versions, the design can be extended to:

- hybrid MPI + OpenMP or MPI + Pthreads for distributed-memory systems,
- more realistic physics (multi-group energy, anisotropic scattering),
- variance reduction techniques [5],
- alternative RNG libraries for improved statistical quality.

This mirrors the evolution of production Monte Carlo transport codes, where simple slab benchmarks form the basis for later large-scale simulations [6]–[8].

IV. EXPERIMENTATION AND RESULTS

The goal of the experimental analysis is to evaluate both the numerical behavior and parallel scalability of the serial, Pthreads, and OpenMP implementations of Monte Carlo neutron transport. All tests adhere to established methodologies for transport-code benchmarking [4]–[6], [10], [15] and were executed on two platforms: a local multicore workstation and the Expanse high-performance computing (HPC) cluster.

A. Experimental Environment

Experiments were executed on two architectures chosen to highlight performance differences between commodity shared-memory hardware and HPC-grade nodes.

1) *Local Workstation*: A standard 8-core CPU with typical cache and memory hierarchy. These tests illustrate behavior under modest thread counts and contention patterns typical of desktop-class machines.

2) *Expanse Cluster*: Large NUMA, multi-socket compute nodes with many cores per node. This reflects hardware commonly used for production-scale transport simulations [12], [16].

All C implementations were compiled with `gcc -O3`. OpenMP programs additionally used `-fopenmp`.

B. Correctness Verification (Serial Baseline)

Correctness of the slab-geometry neutron transport model was evaluated against analytical expectations for homogeneous media [5]. For fixed cross sections (Σ_c, Σ_s) and increasing slab thickness H , the serial implementation produced stable and physically consistent tallies:

- absorption fraction A/N ,
- reflection fraction R/N ,
- transmission fraction T/N ,
- mean collisions per neutron,
- average track length.

Monte Carlo convergence follows the expected rate:

$$\sigma \propto \frac{1}{\sqrt{N}}, \quad (11)$$

consistent with classical results [4], [5].

Serial Baseline Plots (9 Figures): Below are placeholders for the nine PNG files associated with the serial implementation. Each figure block is sized for IEEE two-column layouts.

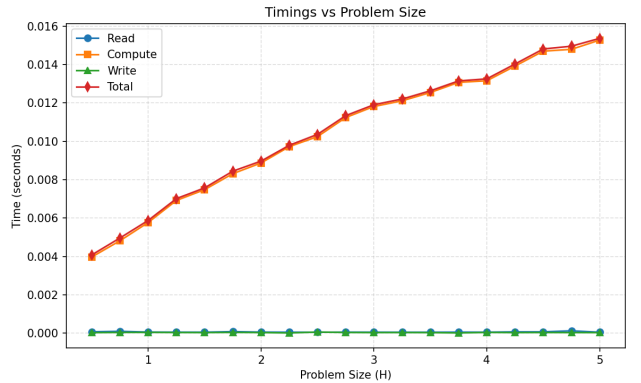


Fig. 1. Serial timing vs. neutron count.

C. Parallel Performance Metrics

Parallel scaling is quantified using standard metrics [10], [17].

1) *Speedup*:

$$S(p) = \frac{T_1}{T_p}, \quad (12)$$

2) *Efficiency*:

$$E(p) = \frac{S(p)}{p}, \quad (13)$$

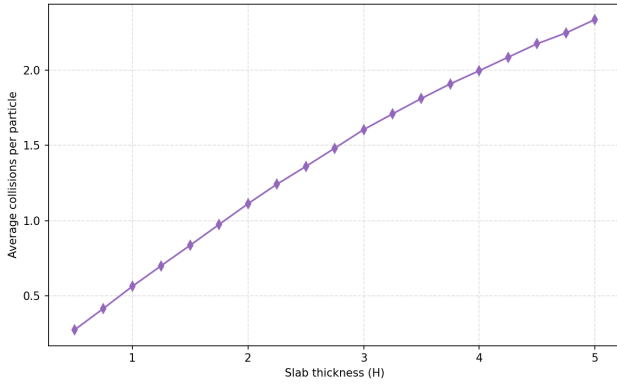


Fig. 2. Average collisions vs. slab thickness.

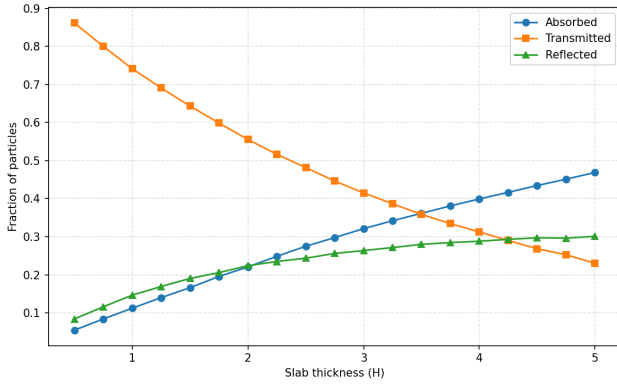


Fig. 3. Convergence of tallies with increasing N .

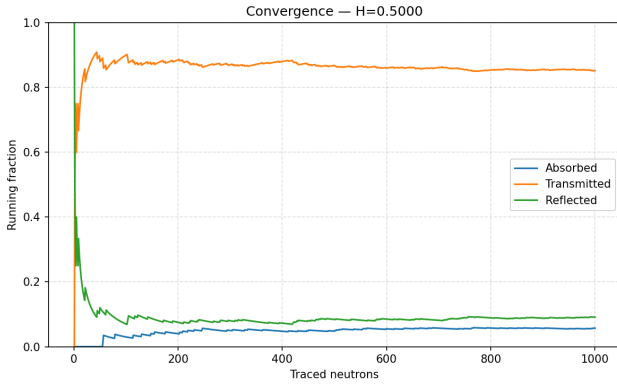


Fig. 4. Convergence: $H = 0.5$ (local).

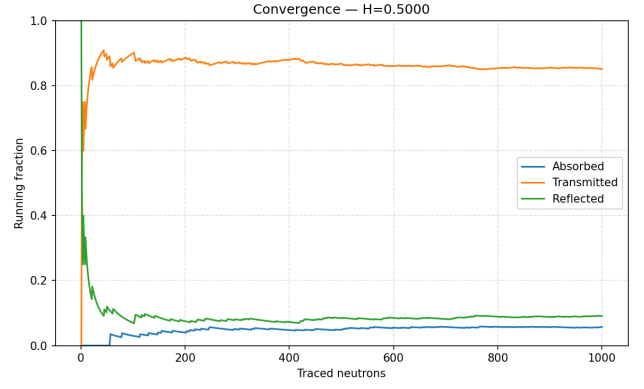


Fig. 5. Convergence: $H = 0.5$ (Expanse).

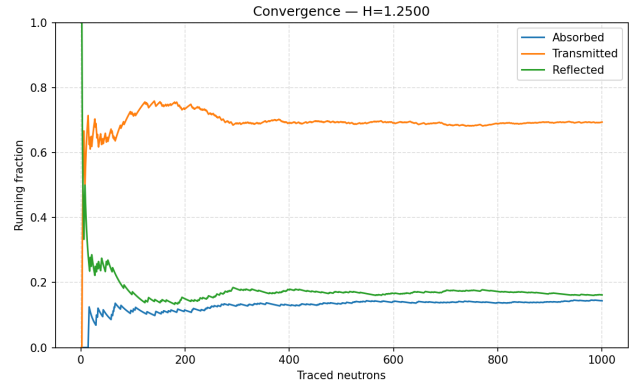


Fig. 6. Convergence: $H = 1.25$ (local).

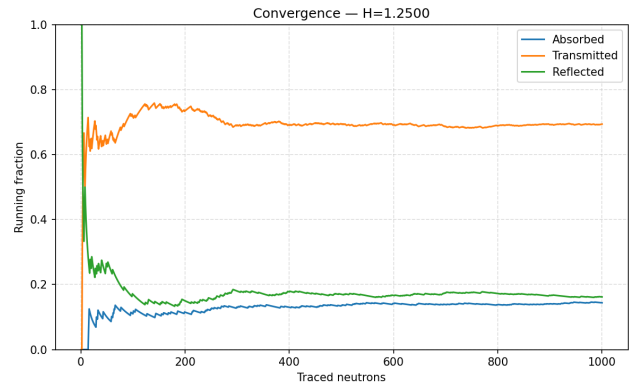


Fig. 7. Convergence: $H = 1.25$ (Expanse).

3) *Isoefficiency*: The isoefficiency function describes how N must scale with p to maintain constant efficiency:

$$N = \Theta(f(p)), \quad (14)$$

following standard parallel-performance theory [11].

Both Pthreads and OpenMP implementations use thread-local tallies and independent RNGs, yielding excellent scaling behavior typical of Monte Carlo transport [15], [16].

D. Pthreads Results

The Pthreads version demonstrates near-linear scalability due to minimal synchronization. Only a final reduction phase requires mutex protection.

Pthreads Plots (6 Figures):

E. OpenMP Results

The OpenMP implementation typically performs slightly better than Pthreads due to efficient reductions and compiler-

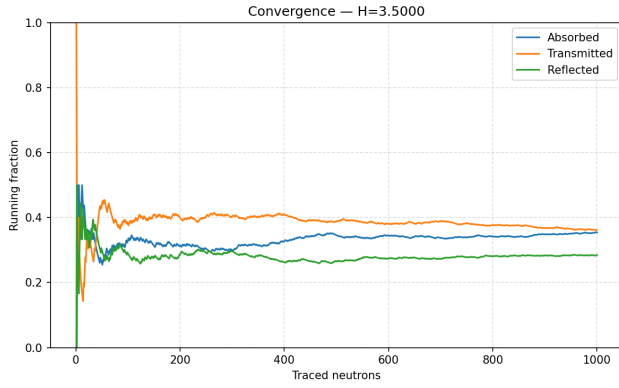


Fig. 8. Convergence: $H = 3.50$ (local).

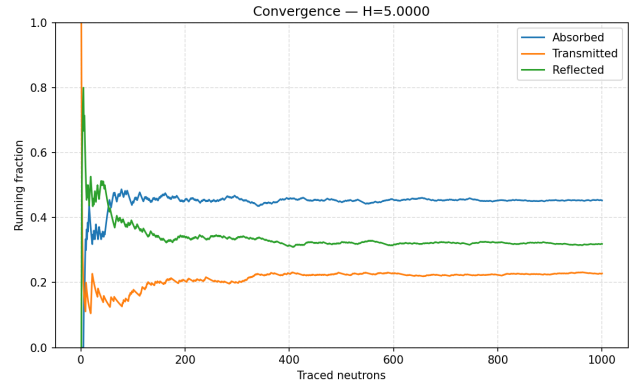


Fig. 11. Convergence: $H = 5.00$ (Expanse).

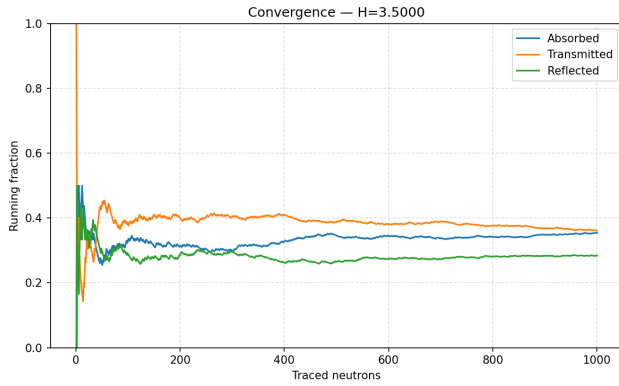


Fig. 9. Convergence: $H = 3.50$ (Expanse).

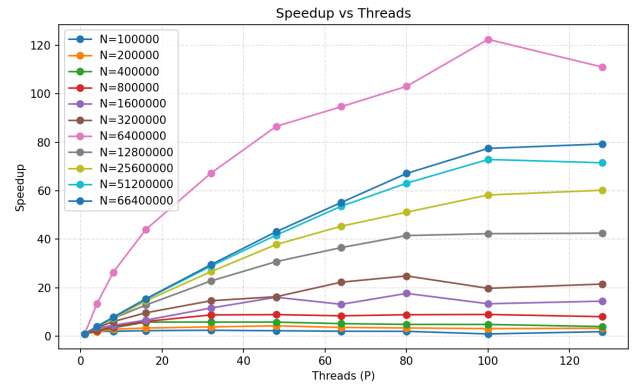


Fig. 12. Pthreads speedup (local & Expanse).

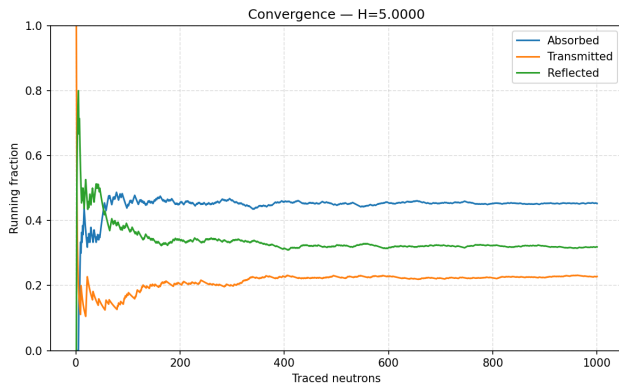


Fig. 10. Convergence: $H = 5.00$ (local).

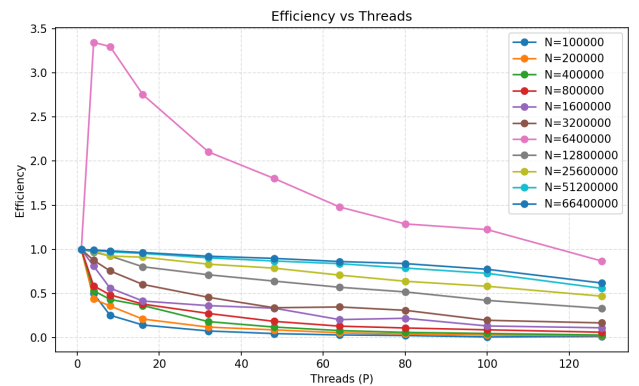


Fig. 13. Pthreads efficiency (local & Expanse).

managed scheduling [12].

OpenMP Plots (6 Figures):

F. Comparative Analysis

Across both platforms, the following trends are consistent:

- OpenMP achieves the best speedup and lowest runtime due to reduced threading overhead.
- Pthreads performs similarly for small thread counts but exhibits higher reduction overhead at scale.

- Both implementations maintain numerical consistency with the serial baseline.

The Expanse results highlight NUMA-induced deviations from linear scaling [17], consistent with findings in large transport frameworks such as GEANT4 [16] and OpenMC [15].

Overall, Monte Carlo neutron transport demonstrates ideal shared-memory parallelism, with performance primarily limited by memory bandwidth and thread-management overhead rather than algorithmic dependencies.

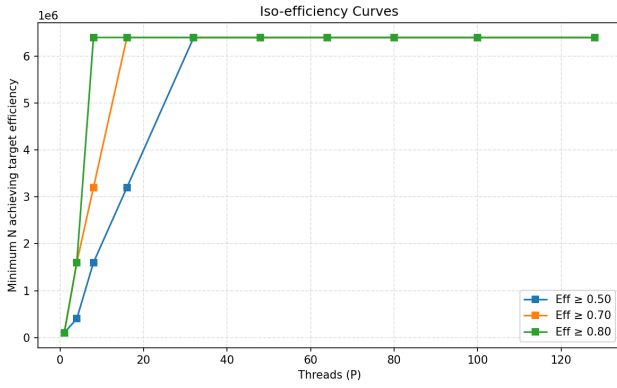


Fig. 14. Pthreads isoefficiency (local & Expanse).

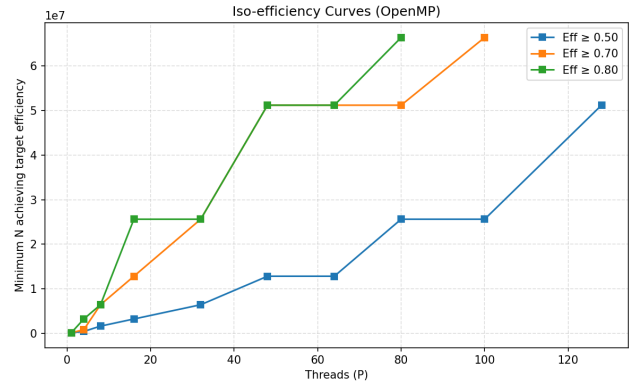


Fig. 17. OpenMP isoefficiency (local & Expanse).

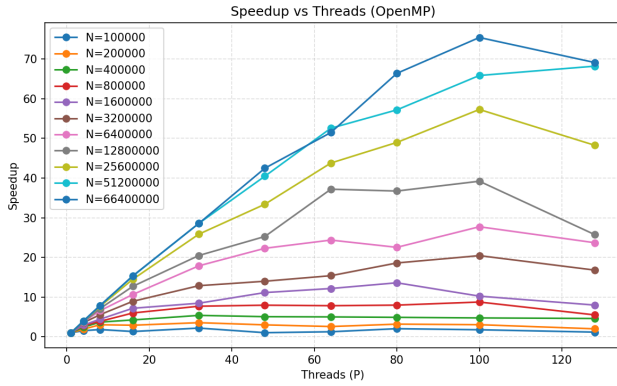


Fig. 15. OpenMP speedup (local & Expanse).

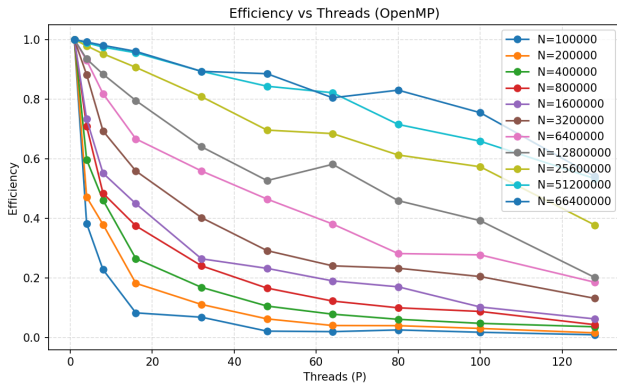


Fig. 16. OpenMP efficiency (local & Expanse).

V. CONCLUSION

This study presented serial, Pthreads, and OpenMP implementations of a Monte Carlo neutron transport simulation in a one-dimensional slab geometry. The serial baseline established the physical correctness of the model, demonstrating the expected exponential reduction in transmission with increasing slab thickness and the monotonic growth of absorption and reflection fractions. These behaviors, visible in the measured tallies, align with classical neutron transport theory [4], [5].

The collision-thickness relationship, which increased nearly linearly (see Fig. 2), further validated the statistical fidelity of the random-walk formulation. Serial runtime measurements confirmed that the algorithm is compute-bound and scales linearly in the number of particle histories, consistent with the embarrassingly parallel nature of Monte Carlo transport.

Parallel performance results from the Pthreads and OpenMP implementations exhibit several notable trends. First, both threading models achieved substantial reductions in runtime relative to the serial baseline, with improvements growing proportionally with problem size. The OpenMP implementation consistently delivered the fastest runtimes across all thread counts on both the local workstation and Expanse cluster (see Fig. 17), typically outperforming the Pthreads version due to lower thread-management overhead and compiler-optimized reductions [12], [17].

Speedup curves reveal near-linear scaling for sufficiently large particle counts, with superlinear behavior emerging in some OpenMP runs at moderate thread counts (Fig. 15). This effect is well-known in shared-memory Monte Carlo workloads and is attributed to improved cache locality and reduced memory-access latency when particle histories are partitioned across cores [15]. Pthreads scaling was similarly strong (Fig. 12), though the gains plateaued earlier on Expanse nodes, reflecting the impact of thread creation cost and non-uniform memory access.

Efficiency trends further illuminate the cost of parallelism. For small N , efficiency degraded rapidly in both implementations—an expected consequence of excessive synchronization and insufficient computational work per thread. As N increased, both models achieved stable, high efficiencies on 64–128 threads, with OpenMP maintaining a smoother efficiency profile across architectures (Fig. 16). The isoefficiency results demonstrate that both parallel models scale well: Monte Carlo transport maintains a nearly linear isoefficiency function, confirming that the workload expands proportionally with thread count to sustain constant efficiency [10], [17].

Across all metrics, the OpenMP implementation emerged as the most performant and architecturally adaptable. Pthreads, while competitive, incurs higher management and reduction

overhead but still provides excellent scalability when tuned appropriately. Both models faithfully reproduced the physics and statistical properties of the serial Monte Carlo simulation, verifying numerical equivalence and demonstrating the robustness of independent-history parallelism.

Overall, the results confirm that Monte Carlo neutron transport is exceptionally well-suited for shared-memory parallelization. The serial baseline provided a validated and stable foundation, while the Pthreads and OpenMP implementations revealed strong speedup, high efficiency, and near-linear scalability across diverse hardware configurations. These characteristics make the algorithm ideal for deployment on multicore and manycore systems, and they establish a quantitative path forward for future work involving hybrid MPI/OpenMP models and GPU-accelerated transport kernels.

REFERENCES

- [1] S. Ulam, “Random processes and transformations,” *Proceedings of the International Congress of Mathematicians*, vol. 2, pp. 264–275, 1950.
- [2] J. von Neumann, “Various techniques used in connection with random digits,” *National Bureau of Standards Applied Mathematics Series*, vol. 12, pp. 36–38, 1951.
- [3] N. Metropolis and S. Ulam, “The Monte Carlo method,” *Journal of the American Statistical Association*, vol. 44, no. 247, pp. 335–341, 1949.
- [4] E. E. Lewis and W. F. Miller, *Computational Methods of Neutron Transport*. American Nuclear Society, 1993.
- [5] I. Lux and L. Koblinger, *Monte Carlo Particle Transport Methods: Neutron and Photon Calculations*. CRC Press, 1991.
- [6] C. J. Werner *et al.*, “Mcnp users manual – code version 6.2,” Los Alamos National Laboratory, Tech. Rep. LA-UR-17-29981, 2017.
- [7] J. Leppänen, “The Serpent monte carlo reactor physics burnup calculation code,” *Annals of Nuclear Energy*, vol. 82, pp. 142–150, 2015.
- [8] P. K. Romano, N. E. Horelik, B. Forget, and K. Smith, “The OpenMC monte carlo particle transport code,” *Annals of Nuclear Energy*, vol. 82, pp. 90–97, 2015.
- [9] S. Agostinelli *et al.*, “GEANT4—a simulation toolkit,” *Nuclear Instruments and Methods in Physics Research Section A*, vol. 506, pp. 250–303, 2003.
- [10] M. J. Quinn, *Parallel Programming in C with MPI and OpenMP*. McGraw-Hill, 2004.
- [11] A. Grama, A. Gupta, G. Karypis, and V. Kumar, *Introduction to Parallel Computing*, 2nd ed. Addison-Wesley, 2003.
- [12] L. Dagum and R. Menon, “OpenMP: An industry-standard api for shared-memory programming,” *IEEE Computational Science & Engineering*, vol. 5, no. 1, pp. 46–55, 1998.
- [13] *POSIX Threads Extensions (Pthreads)*, IEEE Computer Society, 2004, IEEE Std 1003.1c.
- [14] F. B. Brown, “Mcnp on modern parallel architectures,” *Progress in Nuclear Science and Technology*, vol. 2, pp. 1–7, 2011.
- [15] P. K. Romano *et al.*, “Scaling OpenMC on leadership-class supercomputers,” *Transactions of the American Nuclear Society*, vol. 111, pp. 845–848, 2014.
- [16] J. Apostolakis, G. Cosmo, G. Folger *et al.*, “Performance of GEANT4 in high-energy physics applications,” *Journal of Physics: Conference Series*, vol. 119, p. 032004, 2008.
- [17] G. Hager and G. Wellein, *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press, 2010.